

Σε αυτή τη σειρά ασκήσεων θα υλοποιήσετε το Παιχνίδι της Ζωής (Game of Life) του John Conway. Το παιχνίδι αυτό παίζεται από έναν παίκτη, ο οποίος τοποθετεί ζωντανούς οργανισμούς στον αρχικό πίνακα του παιχνιδιού και από εκεί και πέρα παρακολουθεί την εξέλιξη του παιχνιδιού από βήμα σε βήμα (γενιά σε γενιά).

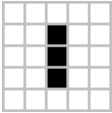
Το παιχνίδι αποτελείται από ένα τετραγωνικό πίνακα $n \times n$ κελιών, όπου κάθε κελί μπορεί να καταλαμβάνεται από έναν το πολύ ζωντανό οργανισμό. Στην περίπτωση αυτή λέμε ότι το κελί αυτό είναι "ζωντανό" (alive).

Σε κάθε νέα γενιά, μπορεί να γεννηθούν νέοι οργανισμοί και να πεθάνουν ή να επιβιώσουν οι παλαιότεροι. Αν ένα κελί θα είναι ζωντανό ή όχι στην επόμενη γενιά εξαρτάται από το πλήθος των γειτονικών κελιών (τα κελιά ακριβώς πάνω, κάτω, δεξιά, αριστερά μαζί με τα 4 διαγώνια) που είναι ζωντανά στην προηγούμενη γενιά. (Κελιά που βρίσκονται στην άκρη του πίνακα έχουν λιγότερα γειτονικά.)

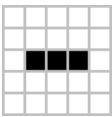
Ο πίνακας της επόμενης γενιάς υπολογίζεται από τον πίνακα της προηγούμενης γενιάς βάσει των εξής κανόνων:

1. Ένα ζωντανό κελί με 0 ή 1 ζωντανούς οργανισμούς στα γειτονικά κελιά, πεθαίνει στην επόμενη γενιά.
2. Ένα ζωντανό κελί με 2 ή 3 ζωντανούς οργανισμούς στα γειτονικά κελιά, επιβιώνει στην επόμενη γενιά.
3. Ένα ζωντανό κελί με 4 ή παραπάνω ζωντανούς οργανισμούς στα γειτονικά κελιά, πεθαίνει στην επόμενη γενιά.
4. Σε ένα μη ζωντανό κελί με 3 ακριβώς γειτονικούς ζωντανούς οργανισμούς, γεννιέται ένας (ζωντανός) οργανισμός στην επόμενη γενιά.

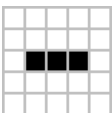
Για παράδειγμα, θεωρήστε ένα παιχνίδι της ζωής που διεξάγεται σε έναν 5×5 πίνακα όπου αρχικά έχουν τοποθετηθεί 3 ζωντανοί οργανισμοί όπως φαίνεται στο παρακάτω σχήμα όπου τα ζωντανά κελιά είναι αυτά με μαύρο χρώμα.



Η εφαρμογή των κανόνων δίνει τον επόμενο πίνακα:



Ο πίνακας της επόμενης γενιάς συμπίπτει με τον αρχικό, οπότε αυτό το παιχνίδι της ζωής εξελίσσεται με μια ατέρμονη εναλλαγή μεταξύ των δύο πινάκων:



Το Παιχνίδι της Ζωής, αναλόγως με την τοποθέτηση ζωντανών οργανισμών στον αρχικό πίνακα, μπορεί να δώσει πολύπλοκες συμπεριφορές στην εξέλιξη του, πχ., σχηματισμούς ζωντανών οργανισμών που μπορούν να αναπαράγονται και στη συνέχεια οι "γονείς" να καταστρέφονται. Περισσότερες πληροφορίες μπορείτε να βρείτε στο λήμμα Game of Life της Wikipedia.

Αρχείο άσκησης

Τη Σειρά Ασκήσεων 4 συνοδεύει το module `game_of_life.py`. Στο αρχείο αυτό βρίσκονται ορισμοί συναρτήσεων που υλοποιούν το Παιχνίδι της Ζωής, χωρίς κώδικα στο σώμα τους.

Τρόπος παράδοσης

Θα παραδώσετε το αρχείο `game_of_life.py`, αφού συμπληρώσετε τον κώδικα που σας ζητείται.

1. Οι φοιτητές που έχουν εγγραφεί στο eclass θα παραδώσουν την εργασία από εκεί.
2. Οι φοιτητές που δεν έχουν εγγραφεί στο eclass θα την παραδώσουν με email στο dimakis@aueb.gr με θέμα (subject): [2022 - ΑΣΚΗΣΗ 4] (με τις αγκύλες) που επίσης περιέχει το ονοματεπώνυμο τους στο σώμα του μηνύματος και σε σχόλιο μέσα στο `game_of_life.py`.

Εάν δεν απαντήσετε κάποιες ασκήσεις, δεν χρειάζεται να διαγράψετε ή να τοποθετήσετε εντός σχολίων τις αντίστοιχες συναρτήσεις. Μπορείτε να τις αφήσετε όπως είναι - χωρίς να τις αλλάξετε.

Άσκηση 1

Δώστε τον κώδικα του κατασκευαστή του σύνθετου δεδομένου `board` το οποίο αναπαριστά τον τετραγωνικό πίνακα του παιχνιδιού. Η κλήση `board(n)` επιστρέφει αναπαράσταση του πίνακα, η οποία είναι λεξικό (αντικείμενο της τάξης `dict`) με $n*n$ στοιχεία. Κάθε κελί αντιστοιχεί σε ένα στοιχείο του λεξικού με `key` το tuple `(i, j)` το οποίο δηλώνει τις συντεταγμένες του, και τιμή `True` ή `False` αναλόγως αν το κελί είναι ζωντανό ή όχι. Η αρίθμηση της γραμμής `i` και στήλης `j` που βρίσκεται ένα κελί είναι ένας ακέραιος από 0 έως $n-1$.

Στον πίνακα παιχνιδιού που επιστρέφει η `board` κανένα κελί αρχικά δεν είναι ζωντανό, οπότε οι τιμές όλων των στοιχείων είναι `False`.

```
def board(n):
    """Kataskeuastis board (pinaka paixnidιου).

    n -- parametros diastasis pinaka

    Kataskeuazei anaparastasi pinaka paixnidιου me n x n kelia, opou
    kanena keli den einai zwntano arxika.

    O pinakas anaparistatai ws le3iko (dict) me n * n stoixeia.
    Ka8e keli antistoixei se ena stoixeio me key (kleidi) to tuple (i,j), opou
    i o ari8mos grammis kai j o ari8mos stilis pou brisketai to keli.
    (H ari8misi grammwn kai sthlwn einai apo 0 ews n-1. To panw aristera keli
    brisketai sto (0,0).)
    H timi (value) tou stoixeiou einai True 'h False analoga
    ean to keli einai zwntano 'h oxi.

    Paradeigmata:

    >>> game = board(3)
    >>> len(game)
    9
    >>> game == {(0, 0): False, (0, 1): False, (0, 2): False, (1, 0): False, (1, 1): False, (1, 2): False, (2, 0): False, (2, 1): False, (2, 2): Fal
    True
    >>> game[2,1]
    False
    """
    """GRAPSTE TON KWDIKA SAS EDW."""
```

Άσκηση 2

Δώστε τον κώδικα των συναρτήσεων `is_alive`, `set_alive` και `get_size` οι οποίες λειτουργούν ως επιλογείς των δεδομένων τύπου `board`.

Η `is_alive(board, p)` επιστρέφει τη λογική τιμή `True` ή `False` αναλόγως εάν το κελί που βρίσκεται στην τοποθεσία `p` του πίνακα παιχνιδιού `board`, είναι ζωντανό ή όχι. Η τοποθεσία `p` είναι ένα tuple της μορφής `(i,j)`, όπου `i` και `j` είναι ο αριθμός γραμμής και στήλης αντίστοιχα του πίνακα παιχνιδιού.

```
def is_alive(board, p):
    """Elegxei ean ena keli einai zwntano.

    board -- o pinakas paixnidιου opou brisketai to keli
    p -- h 8esh tou keliou.

    To orisma p einai tuple tis morfis (i,j).
    Epistrefei True ean to keli einai zwntano, alliws False.

    Paradeigma:

    >>> is_alive(board(4), (3,2))
    False
    """
    """GRAPSTE TON KWDIKA SAS EDW."""
```

Η `set_alive(board, p, alive)` ορίζει το κελί στη θέση `p` (tuple της μορφής `(i,j)`) του πίνακα `board` ως ζωντανό ή όχι αναλόγως εάν η λογική τιμή `alive` είναι `True` ή `False`.

```
def set_alive(board, p, alive):
    """Topo8etei 'h afairei zwi apo ena keli.

    board -- o pinakas paixnidιου opou brisketai to keli
    p -- h 8esh tou keliou (tuple tis morfis (i,j))
    alive -- logiki timi.

    To keli ginetai zwntano ean h alive einai True. An h alive einai False,
    to keli pe8ainei.

    Paradeigmata:

    >>> game = board(4)
    >>> is_alive(game, (3,2))
    False
    >>> set_alive(game, (3,2), True)
    >>> is_alive(game, (3,2))
    True
    >>> set_alive(game, (3,2), False)
    >>> is_alive(game, (3,2))
    False
    """
    """GRAPSTE TON KWDIKA SAS EDW."""
```

Η `get_size(board)` επιστρέφει τη διάσταση του πίνακα παιχνιδιού `board` ως τιμή τύπου `int`. Ένας πίνακας με `n x n` κελιά έχει διάσταση `n`.

```
def get_size(board):
    """Mege8os pinaka paixnidιου.

    board -- pinakas paixnidιου.

    Epistrefei n (timi int) ean to board anaparista pinaka paixnidιου n x n.

    Paradeigmata:

    >>> get_size(board(4))
    4
    >>> get_size(board(10))
    10
    """
    """GRAPSTE TON KWDIKA SAS EDW."""
```

Άσκηση 3

Δώστε τον κώδικα της συνάρτησης `copy_board` η οποία επιστρέφει δεδομένο `board` το οποίο είναι αντίγραφο του δεδομένου `board` που δίνεται στο όρισμα.

```
def copy_board(board):
    """Kataskeui antigrafou pinaka paixnidιου.

    board -- pinakas paixnidιου.

    Epistrefei ena neo pinaka paixnidιου pou einai antigrafo tou board.

    Paradeigmata:

    >>> game = board(10)
    >>> set_alive(game, (4,7), True)
    >>> game2 = copy_board(game)
    >>> game2 is game
    False
    >>> is_alive(game2, (4,7))
    True
    """
    """GRAPSTE TON KWDIKA SAS EDW."""
```

Άσκηση 4

Δώστε τον κώδικα της συνάρτησης `get_iterator` η οποία επιστρέφει ένα iterator για σάρωση των κελιών ενός πίνακα παιχνιδιού. Η κλήση `get_iterator(board)` επιστρέφει έναν iterator ο οποίος δίνει τα κελιά του `board` αρχίζοντας από αυτά της πρώτης γραμμής σε αύξουσα σειρά στήλης, ακολουθούμενα από τα κελιά της 2ης γραμμής σε αύξουσα σειρά στήλης, ακολουθούμενα από τα κελιά της 3ης γραμμής κτλ.

Προσέξτε ότι η σειρά με την οποία σαρώνονται τα κελιά θα πρέπει να είναι όπως περιγράφεται παραπάνω, ανεξάρτητα από τη σειρά με την οποία έχουν εισαχθεί στο λεξικό που αναπαριστά τον πίνακα παιχνιδιού (στη συνάρτηση `board`). (Στις εκδόσεις της Python πριν την 3.5, η σειρά που επιστρέφονται τα στοιχεία ενός λεξικού `d` από έναν iterator, πχ που έχει κατασκευαστεί με την κλήση `iter(d)`, εν γένει διαφέρει από τη σειρά εισαγωγής τους στο λεξικό κατά την κατασκευή του `d`. Από την έκδοση 3.5 ορίζεται ότι η σειρά επιστροφής στοιχείων είναι η ίδια με τη σειρά εισαγωγής. Επειδή ο κώδικας σας θα πρέπει να δουλεύει σε παλαιότερες εκδόσεις της Python ή και σε μελλοντικές όπου μπορεί να ξανα-αλλάξει η σειρά επιστροφής στοιχείων, θα πρέπει να δίνει τα στοιχεία με τη σωστή σειρά *ανεξάρτητα από τον τρόπο εισαγωγής*.)

Για κάθε κελί ο iterator επιστρέφει ένα tuple με δύο στοιχεία. Στο πρώτο περιέχει τη θέση του κελιού δοσμένη ως tuple της μορφής `(i,j)` (όπως στη συνάρτηση `is_alive`) και στο δεύτερο στοιχείο μια λογική τιμή που είναι `True` ή `False` αναλόγως αν το κελί είναι ζωντανό ή όχι.

```
def get_iterator(board):
    """Iterator gia sarwsi stoixeiwn pinaka paixnidιou.

    board -- pinakas paixnidιou.

    Epistrefei iterator pou dinei ta kelia tou board arxizontas apo
    ta kelia tis grammis 0: (0,0), (0,1), (0,2),..., meta akolou8oun ta
    kelia tis grammis 1: (1,0), (1,1), (1,2),... ktl. mexri na e3antli8oun
    ola ta kelia. Gia ka8e keli, o iterator epistrefei tin 8esi tou kai
    logiki timi True 'h False analogws ean einai zwntano 'h oxi.

    Paradeigma:

    >>> game = board(3)
    >>> set_alive(game, (2, 1), True)
    >>> for cell in get_iterator(game):
    ...     print(cell)
    ...
    ((0, 0), False)
    ((0, 1), False)
    ((0, 2), False)
    ((1, 0), False)
    ((1, 1), False)
    ((1, 2), False)
    ((2, 0), False)
    ((2, 1), True)
    ((2, 2), False)
    """
    """GRAPSTE TON KWDIKA SAS EDW."""
```

Άσκηση 5

Δώστε τον κώδικα της συνάρτησης `print_board` που εμφανίζει τον πίνακα παιχνιδιού που δίνεται ως όρισμα. Χρησιμοποιήστε τον χαρακτήρα █ (κωδικός unicode: 11035) ή 'x' αν δεν εμφανίζεται σωστά στο τερματικό σας ο χαρακτήρας █ για ζωντανά κελιά και τον χαρακτήρα □ (κωδικός unicode: 11036) ή κενό (space) ' ' αν δεν εμφανίζεται σωστά, για μη ζωντανά. (Η κλήση στην ενσωματωμένη συνάρτηση `chr(x)` επιστρέφει ως string τον χαρακτήρα με unicode κωδικό x.)

Τα κελιά κάθε γραμμής του πίνακα παιχνιδιού εμφανίζονται στην ίδια σειρά, αρχίζοντας από αριστερά προς δεξιά σε αύξουσα σειρά στήλης του πίνακα. Οι γραμμές εμφανίζονται σε διαδοχικές σειρές με αύξουσα σειρά. Συνεπώς, το πάνω αριστερά κελί είναι το (0,0).

```
def print_board(board):
    """Emfanizei pinaka paixnidιou.

    board -- pinakas paixnidιou.

    Emfanizei ton pinaka paixnidιou board opou me mauro tetragwno
    (xaraktiras unicode 11035 'h 'x' an den emfanizetai swsta)
    dinontai ta zwntana kelia, kai me aspro (xaraktiras unicode 11036 'h
    keno ' ' an den emfanizetai swsta) ta pe8amena.
    To panw aristera keli einai auto sti 8esi (0,0).

    Paradeigma:

    >>> game = board(5)
    >>> set_alive(game, (0,0), True)
    >>> set_alive(game, (2,2), True)
    >>> set_alive(game, (4,4), True)
    >>> set_alive(game, (3,4), True)
    >>> set_alive(game, (0,4), True)
    >>> print_board(game)
    █ □ □ □ █
    □ □ □ □ □
    □ □ █ □ □
    □ □ □ □ █
    □ □ □ □ █
    """
    """GRAPSTE TON KWDIKA SAS EDW."""
```

Σημειώστε ότι ο τρόπος εμφάνισης του χαρακτήρα █ και □ στον υπολογιστή σας μπορεί να διαφέρει από αυτόν που δίνεται εδώ γιατί εξαρτάται από τη συμβολοσειρά που χρησιμοποιεί το τερματικό σας. (Για να εμφανίσετε τους χαρακτήρες █□ από τη γραμμή εντολών του MSDOS κάντε τα εξής:

- Επιλέξτε μια από τις γραμματοσειρές (δεξί click στην πάνω αριστερή γωνία του παραθύρου του MSDOS -> Ιδιότητες -> Γραμματοσειρά): MS Gothic, N SimSun, SimSun-ExtB.
- Στη γραμμή εντολών γράψτε: `chcp 65001`

Τώρα θα πρέπει να εμφανίζονται κανονικά οι χαρακτήρες. (Για να δοκιμάσετε, κάντε copy/paste τους χαρακτήρες παραπάνω.)

Μην χρησιμοποιήσετε διαφορετικούς χαρακτήρες που ίσως οπτικά μοιάζουν περισσότερο. Η ορθότητα του κώδικά σας θα ελεγχθεί χρησιμοποιώντας τους κωδικούς unicode των δύο χαρακτήρων και όχι την οπτική τους εμφάνιση.

Άσκηση 6

Δώστε τον κώδικα της συνάρτησης `neighbors` που επιστρέφει τις τοποθεσίες των γειτονικών κελιών του κελιού που δίνεται στο όρισμα. Η κλήση `neighbors(p)` όπου `p` ένα tuple της μορφής `(i,j)` επιστρέφει ένα σύνολο (αντικείμενο τάξης `set`) το οποίο περιέχει 8 tuples που αντιστοιχούν στις τοποθεσίες των γειτονικών κελιών του `(i,j)`. Τα γειτονικά κελιά είναι αυτά που βρίσκονται πάνω, κάτω, δεξιά, αριστερά και μια διαγώνια θέση από το `(i,j)`. Επιστρέφονται και τα 8 γειτονικά κελιά ανεξαρτήτως αν κάποια από αυτό βρίσκονται εκτός πίνακα ή έχουν αρνητικό αριθμό γραμμής ή στήλης. Στα γειτονικά κελιά δεν ανήκει το ίδιο το κελί `(i,j)` που δίνεται στο όρισμα της `neighbors`.

```
def neighbors(p):
    """Geitonika kelia.

    p -- 8esh keliou (tuple tis morfis (i,j)).

    Epistrefei synolo (set) pou perioxontai oi 8eseis twn 8 geitonikwn
    keliwn tou p. Sto epistrefomeno synolo den periexetai to keli p.

    Paradeigmata:

    >>> neighbors((3,2)) == {(3, 3), (3, 1), (2, 1), (2, 3), (4, 3), (2, 2), (4, 2), (4, 1)}
    True
    >>> neighbors((0,0)) == {(0, 1), (-1, 1), (-1, 0), (-1, -1), (0, -1), (1, 0), (1, -1), (1, 1)}
    True
    >>> neighbors((0,10)) == {(-1, 9), (1, 10), (-1, 11), (0, 11), (-1, 10), (1, 9), (0, 9), (1, 11)}
    True
    """
    """GRAPSTE TON KWDIKA SAS EDW."""
```

Άσκηση 7

Δώστε τον κώδικα των συναρτήσεων `place_blinker` και `place_glider` οι οποίες τοποθετούν ζωντανούς οργανισμούς σε κελιά σε διάφορους σχηματισμούς. (Περισσότερους σχηματισμούς μπορείτε να δείτε στο λήμμα `Game of Life` της Wikipedia.)

Η κλήση `place_blinker(board, p)` τοποθετεί ζωντανούς οργανισμούς στον πίνακα `board` σύμφωνα με τον σχηματισμό `blinker`, αρχίζοντας από το κελί `p`. Εάν παραληφθεί το `p`, λαμβάνει την προκαθορισμένη τιμή `(0,0)`.

```
def place_blinker(board, p = (0,0)):
    """Topo8etisi blinker.

    board -- pinakas paixnidιou.
    p -- keli topo8etisis (tuple (i,j) me proka8orismeni timi (0,0))

    Topo8etei 3 zwntanous organismous sto board
    se geitonika kelia tis idias stilis, arxizontas apo ti 8esi p kai
    proxwrontas stis parakatw grammes.

    Paradeigmata:

    >>> game = board(5)
    >>> place_blinker(game)
    >>> print_board(game)
    █ □ □ □ □
    █ □ □ □ □
    █ □ □ □ □
    □ □ □ □ □
    □ □ □ □ □
    >>> place_blinker(game, (1,2))
    >>> print_board(game)
    █ □ □ □ □
    █ □ █ □ □
    █ □ █ □ □
    □ □ █ □ □
    □ □ □ □ □
    >>> place_blinker(game, (4,4))
    >>> print_board(game)
    █ □ □ □ □
    █ □ █ □ □
    █ □ █ □ □
    □ □ █ □ □
    □ □ □ █
    """
    """GRAPSTE TON KWDIKA SAS EDW."""
```

Η κλήση `place_glider(board, p)` τοποθετεί ζωντανούς οργανισμούς σε κελιά του πίνακα `board`, σύμφωνα με τον σχηματισμό `glider` αρχίζοντας από το κελί `p`. Εάν παραληφθεί το `p`, λαμβάνει την προκαθορισμένη τιμή `(0,0)`.

```
def place_glider(board, p = (0,0)):
    """Topo8etisi glider.

    board -- pinakas paixnidιou.
    p -- keli topo8etisis (tuple (i,j) me proka8orismeni timi (0,0))

    Topo8etei 5 zwntanous organismous sto board se sximatismo glider
    arxizontas apo tin topo8esia p, opws fainetai sta parakatw paradeigmata.
    Simeiwste oti to idio to keli p den einai zwntano.

    Paradeigmata:

    >>> game = board(7)
    >>> place_glider(game)
    >>> print_board(game)
    □ □ ■ □ □ □ □
    ■ □ ■ □ □ □ □
    □ ■ ■ □ □ □ □
    □ □ □ □ □ □ □
    □ □ □ □ □ □ □
    □ □ □ □ □ □ □
    □ □ □ □ □ □ □
    □ □ □ □ □ □ □
    >>> place_glider(game, (3,3))
    >>> print_board(game)
    □ □ ■ □ □ □ □
    ■ □ ■ □ □ □ □
    □ ■ ■ □ □ □ □
    □ □ □ □ □ ■ □
    □ □ □ □ ■ □ □
    □ □ □ □ ■ □ □
    □ □ □ □ ■ □ □
    □ □ □ □ ■ □ □
    □ □ □ □ ■ □ □
    """
    """GRAPSTE TON KWDIKA SAS EDW."""
```

Άσκηση 8

Δώστε τον κώδικα της συνάρτησης tick η οποία προχωράει τον πίνακα παιχνιδιού στην επόμενη γενιά. Η κλήση tick(board) αλλάζει τον πίνακα παιχνιδιού board ώστε να αντιστοιχεί στην επόμενη γενιά σύμφωνα με τους κανόνες του Παιχνιδιού της Ζωής.

```
def tick(board):
    """Proxwraei to paixnidi kata ena bima stin epomeni genea.

    board -- pinakas paixnidιou.

    Allazei ton pinaka board proxwrontas mia genea,
    symfwna me tous kanones tou Game of Life.

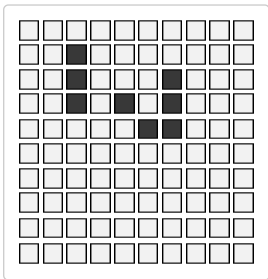
    Paradeigma:

    >>> game = board(6)
    >>> place_glider(game)
    >>> place_blinker(game, (3,4))
    >>> print_board(game)
    □ □ ■ □ □ □
    ■ □ ■ □ □ □
    □ ■ ■ □ □ □
    □ □ □ □ ■ □
    □ □ □ □ ■ □
    □ □ □ □ ■ □
    >>> tick(game)
    >>> print_board(game)
    □ ■ □ □ □ □
    □ □ ■ □ □ □
    □ ■ ■ □ □ □
    □ □ □ □ □ □
    □ □ □ □ ■ □
    □ □ □ □ ■ □
    """
    """GRAPSTE TON KWDIKA SAS EDW."""
```

Άσκηση 9

Συμπληρώστε τον κώδικα στο μπλοκ εντολών της if __name__ == '__main__' που υπάρχει στο τέλος του game_of_life.py. (Δεν βρίσκεται μέσα σε συνάρτηση.)

Εμφανίζονται οι πίνακες παιχνιδιού για τις 100 πρώτες γενιές με αρχικό πίνακα τον:



Οι πίνακες εμφανίζονται ο ένας κάτω από τον άλλον με τουλάχιστον δύο κενές γραμμές μεταξύ τους. Μπορείτε να χρησιμοποιήσετε την ενσωματωμένη συνάρτηση `sleep` (βρίσκεται στο module `time`) για να προσθέσετε μια μικρή καθυστέρηση μεταξύ της εμφάνισης των γενεών. Η κλήση `sleep(x)` "παγώνει" την εκτέλεση κατά x δευτερόλεπτα.

```
if __name__ == '__main__':
    """Paizei to paixnidi gia mia sygkekrimeni arxiki topoθetisi, gia
    100 genies. O pinakas tou paixnidιou emfanizetai se ka8e bima.
    Afiste toulaxiston 2 kenēs grammes anamesa se diadoxikous pinakes.
    """

    # Arxikos pinakas
    game = board(10)
    place_blinker(game, (1,2))
    place_glider(game, (2,4))

    from time import sleep

    """GRAPSTE TON KWDIKA SAS APO KATW."""
```

Ο έλεγχος στην `if` πραγματοποιείται ώστε όταν γίνει `import` του `game_of_life` από το διαλογικό περιβάλλον να μην εκτελεστεί ο κώδικας αυτός. (Σε διαφορετική περίπτωση, θα εκτελούνταν με κάθε εντολή `import` για το `game_of_life`.)

Οι εντολές μπορούν να εκτελεστούν -και να ελέγξετε την ορθότητα του κώδικα σας- όταν τρέξετε το module `game_of_life` έξω από το διαλογικό περιβάλλον, πχ., από τη γραμμή εντολών.

(Η Python αναθέτει το όνομα `__name__` στην τιμή `'__main__'` μόνο στην περίπτωση που το module `game_of_life` έχει εκτελεστεί από τη γραμμή εντολών (πχ, με την εντολή `python game_of_life.py`.)